

List of the techniques used.

Location Retrieval

```
public class GetIP {  
    public static String IPAddress;  
  
    public static String IP() throws MalformedURLException, IOException{  
        String urlString = "http://checkip.amazonaws.com/";  
        URL url = new URL(spec: urlString);  
        try (BufferedReader br = new BufferedReader(new InputStreamReader(in: url.openStream()))){  
            return br.readLine();  
        }  
        catch (IOException ex) {  
  
            String errorMessage = "Error: No Internet Connection.";  
            JOptionPane.showMessageDialog(parentComponent: null, message: errorMessage, title: "No Internet Connection", messageType: JOptionPane.ERROR_MESSAGE);  
            return null;  
        }  
    }  
}
```

‘**GetIP()**’ serves the purpose of obtaining the user's IP address to obtain the approximate location of the user which is vital for the function of this application. For this, the code employs the ‘**BufferedReader**’(‘**br**’) class to efficiently read the resultant data from the URL, which is obtained from an input stream using ‘**url.openStream()**’ which is converted into characters by ‘**InputStreamReader**’ and returned as a string by the ‘**BufferedReader**’ with (**br.readLine()**). It's surrounded by a try block to ensure, the **BufferedReader** is closed after the code is executed.

A catch block is used to catch ‘**IOExceptions**’, and display an error message, indicating the lack of internet connection.

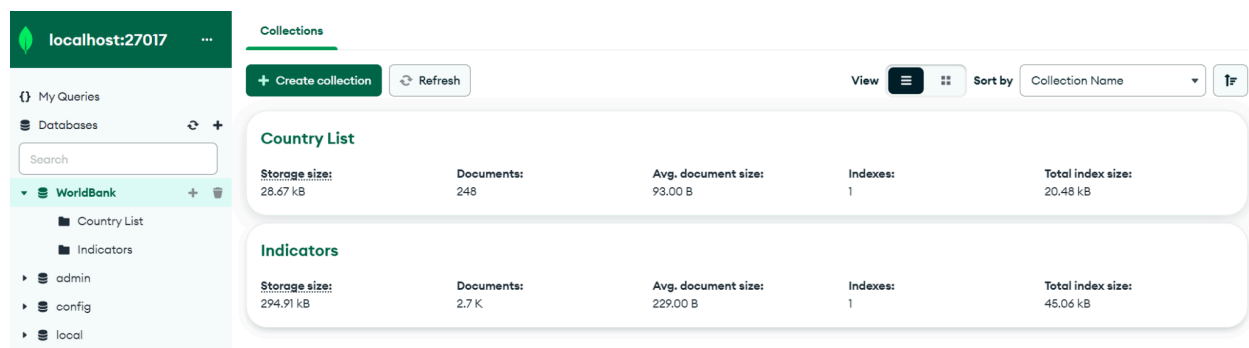
```
public class GetLocation {  
    public static String data() throws IOException, GeoIp2Exception{  
  
        try{  
  
            String IP = GetIP.IP();  
  
            //Loads DataBase of IP and corresponding Country  
            String dblocation = "C:\\Users\\twumn\\OneDrive\\Documents\\GeoLite2-City_20230801\\GeoLite2-City.mmdb";  
            File database = new File(pathname: dblocation);  
            DatabaseReader dbr = new DatabaseReader.Builder(database).build();  
  
            InetAddress ipA =InetAddress.getByName(host: IP);  
            CityResponse response = dbr.city(ipAddress: ipA);  
  
            return response.getCountry().getName();  
        }  
    }  
}
```

getLocation() uses the **GeoIP2 API**, which provides a database that matches, all IPV6 and IPV4 addresses to their corresponding locations. This is useful for obtaining the user's location, which the method returns. I decided to use separate methods to allow for modularity of the application. This design philosophy will be seen throughout the code.

Database

Justification

MongoDB database is used, due to its ability to be expanded. I can easily import the entirety of the World Bank database later on, without any difficulties. It can also query data simply by just adjusting a few filters.



2013: 4.822785479	2013: 608.3863319
2014: 0.566944541	2014: 602.5166003
2015: 2.447563002	2015: 592.4761648
2016: -2.197526466	2016: 590.4273674
2017: 2.403656188	2017: 588.9300803
2018: 2.071348579	2018: 578.9855448
2019: 6.521479845	2019: 584.3865153
2020: 7.821667127	2020: 553.036131
2021: 0.47742893	2021: 426.2294013
2022: ".."	2022: ".."
_id: ObjectId('6530b53297cc9fdfb2368aea')	_id: ObjectId('6530b53297cc9fdfb2368aeb')
Country Name: "Afghanistan"	Country Name: "Afghanistan"
Series Name: "Yearly Inflation"	Series Name: "GDP per capita"

This, how the data is organized, due to the NoSQL nature of MongoDB, it allows me to store data in a non-tabular form.

Connection to data base

```
String url = "mongodb://localhost:27017";
MongoClientURI clientURI = new MongoClientURI(uri:url);
MongoClient mongoClient = new MongoClient(uri:clientURI);
```

MongoDB API allows for easy integration with java.

Retrieving specific data from database

```
public class DataBase {

    public static Object GetData(String collections,String country, String series, String year ) throws IOException, GeoIp2Exception{
        String url = "mongodb://localhost:27017";
        MongoClientURI clientURI = new MongoClientURI(uri:url);
        MongoClient mongoClient = new MongoClient(uri:clientURI);

        MongoDBDatabase mongoDatabase = mongoClient.getDatabase(databaseName: "WorldBank");
        MongoCollection collection = mongoDatabase.getCollection(string: collections);

        Document findQuery = new Document();
        findQuery.put(key:"Country Name",value: country);
        findQuery.put(key:"Series Name",value: series);

        Document result = (Document) collection.find(bson: findQuery).first();

        return result.get(key:year);
    }
}
```

'GetData()' method is designed to query a local Mongo database. A connection is made to the MongoDB local database containing the required World Bank Development Indicators. It accepts the user's country, the name of the indicator, and the year. **'String country'** is used in place of the method **'GetLocation'** country, since the program, will

allow the user to change their country. **'findQuery.put(...)'** adds a filter, which searches through the local database. Finally, the result is filtered by **'result.get(year)'** The result is returned as an **'Object'** because objects would be able to include all data types, which can then be converted to the desired data type.

```
public static Object GetCapital(String country) throws IOException, GeoIp2Exception{
    String uri = "mongodb://localhost:27017";
    MongoClientURI clientURI = new MongoClientURI(uri);
    MongoClient mongoClient = new MongoClient(uri,clientURI);

    MongoDBDatabase mongoDatabase = mongoClient.getDatabase(databaseName: "WorldBank");
    MongoCollection collection = mongoDatabase.getCollection(string: "Country List");

    Document findQuery = new Document();
    findQuery.put(key:"country",value: country);

    Document result = (Document) collection.find(bson: findQuery).first();

    return result.getString(key:"capital");
}
```

'GetCapital()' is a similar method , but this returns the Capital of the user's country.

```
public static String DataString(String collection,String country, String series, String year) throws IOException, GeoIp2Exception{
    Object output = GetData(collections:collection, country, series, year);

    if (output instanceof Number) {
        Number numericValue = (Number) output;
        DecimalFormat decimalFormat = new DecimalFormat(pattern:"#,###.##");
        String outputString = decimalFormat.format(obj:numericValue);
        return outputString;
    }

    if(output == null){
        return "N/A";
    }

    if(output == "."){
        return "N/A";
    }
    else{
        return "N/A";
    }
}
```

'DataString()' uses the output of **'GetData'** and transforms it into a string. It uses the **'Number'** data type, which incorporates all number-based data types, which allows for

easier readability as compared to create if statements each for, **doubles, int, float, etc** data types. It returns “**N/A**” in order for specific conditions such as if data isn’t available to be addressed. This is useful for when generating the table.

Important algorithms

```
public class Dashboard extends javax.swing.JFrame implements ChangeCountryCallback {  
    private String country = GetLocation.data();  
    private String ncountry;  
    private boolean detectcountry = true;  
    private boolean detectedcountry;
```

‘**country**’ and ‘**ncountry**’ are declared, and the user’s country is assigned to the ‘**country**’ variable. ‘**ncountry**’ is used to store a new country the user is in. **boolean** ‘**detectcountry**’ sets is used to set if the program constantly checks for the users country. ‘**detected country**’ states if a new country is detected, so corresponding UI elements can be changed.

```

public void checkCountry(){

    java.util.Timer timer = new java.util.Timer();
    int delay = 0;
    int period = 10000;
    if(detectcountry == true){
        timer.schedule(new TimerTask(){
            @Override
            public void run(){
                if(detectcountry == true){
                    try{
                        String ncountry = GetLocation.data();
                        if(!country.equals(ncountry)){
                            int response = JOptionPane.showConfirmDialog(parentComponent:null, "Do you want to change country to " + ncountry +
                                "?", title: "Confirm", optionType: JOptionPane.YES_NO_OPTION, messageType:JOptionPane.QUESTION_MESSAGE);

                            if ( response == JOptionPane.YES_OPTION){

                                detectedcountry = true;
                                country = GetLocation.data();
                                setText();
                            }

                            else{
                                originalIconC = (ImageIcon) Capl.getIcon(); // Store the original icon
                                hoverIconC = changeIconColor(originalIcon:originalIconC, newColor:Color.GREEN); // Create a new icon with the desired cc
                                Capl.setIcon(icon: hoverIconC);

                                detectcountry = false;
                                detectedcountry = true;
                            }
                        }
                    } catch (IOException ex) {
                        logger.getLogger(name:Dashboard.class.getName()).log(level:Level.SEVERE, msg:null, thrown: ex);
                    } catch (GeoIp2Exception ex) {
                        logger.getLogger(name:Dashboard.class.getName()).log(level:Level.SEVERE, msg:null, thrown: ex);
                    }
                }
            }, delay, period);
        }
    }
}

```

‘**checkCountry()**’ method found in ‘**Dashboard.java**’ utilizes timer-based scheduling, to repeatedly check if the user, is in a new country by using a ‘**java.util.Timer**’ instance to handle scheduling, of a method, extended by ‘**run()**’. Here, it constantly checks the country with the ‘**GetLocation.data()**’; which stores it the country in ‘**ncountry**’ and compares, it with the current country in the ‘**country**’ through an ‘**if statement**’. If the condition is satisfied ‘**!country.equals(ncountry)**’, it brings up a dialog box, to alert the user that they are in a new country, after which it provides an option for the user to change the country or not. If not, a UI element is highlighted, which can be interacted with, to bring up the option again to indicate a new country.

```

public static String MakeFact(ArrayList<String> indicators) throws IOException, GeoIp2Exception{

    Random random = new Random();
    int min = 2013;
    int max = 2022;

    int year = random.nextInt(max-min+1) + min; //Picks a random year between 2013 and 2022

    String yearstr = Integer.toString(year); //Converts randomly picked integer into string
    String country = GetLocation.data(); //Gets the users country

    int size = indicators.size(); //Gets the size of the indicators arraylist
    int indicatorIndex = random.nextInt(size); //Randomizes the indicators Index
    String indicator = indicators.get(indicatorIndex); //Gets the Indicator
    String value = DataString(collection: "Indicators",country, series: indicator, year: yearstr);
    String fact = "Did you know that The " +indicator + " of " +GetLocation.data()+ " in the Year " + year + " was " + value; //Generates a random fact based on the user's country
}

```

‘MakeFact()’ generates a random fact based on the user's country.

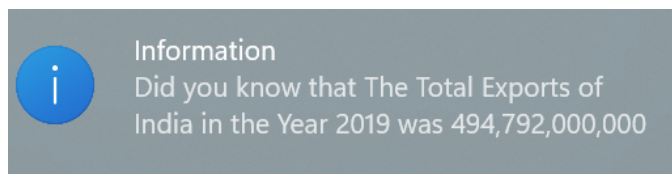


Table and Chart Generation

Table Generation

Initialization of variables for table generation

```

String selectedStartYear = year2.getSelectedItemAt().toString(); // Obtains the Starting Year, from the combobox
String selectedEndYear = year.getSelectedItemAt().toString(); // Obtains the End Year, from the combobox
String selectedIndicator = Indicators2.getSelectedItemAt().toString(); // Obtains the Indicator, from the combobox
String yearstr; // Initializes, the values for the year column
int startyear = Integer.parseInt(selectedStartYear); // Convert the selected years into Integers to be iterated upon
int endyear = Integer.parseInt(selectedEndYear);
int i = 0;
int yeardifference = endyear- startyear; //Used to handle errors with table generation
boolean hasduplicate = false; //Used to handle duplicate columns
boolean endYearError = false; //Used to handle Errors

```

Variables responsible for generating the table and error handling are initialized.

Initializing of Column Data and Retrieval of Table Model

```

ArrayList<String> newColumnData = new ArrayList<>();
String newColumnName = selectedIndicator + "(" + country + ")";

DefaultTableModel tblModel = (DefaultTableModel) jTable.getModel();
int numberOfColumns = tblModel.getColumnCount();
TableColumnModel columnModel = jTable.getColumnModel();

```

Handling of errors

```

//Error Handling with If Statement
if(startyear > endyear){
    JOptionPane.showMessageDialog(parentComponent:null, message:"End year must be greater than the start year!");
}

//Error Handling with If Statement
if(tblModel.getRowCount() != yeardifference + 1 && tblModel.getColumnCount() > 0){
    JOptionPane.showMessageDialog(parentComponent:null, message:"End Year Must be the Same, in order to add different indicators ");
    endYearError = true;
}

//Checks if indicator exists in table
if(tblModel.getColumnCount() > 0){
    for (int l = 0; l < columnModel.getColumnCount(); l++) {
        TableColumn column = columnModel.getColumn(columnIndex:l);
        String columnName = (String) column.getHeaderValue();
        if(newColumnName.equals(columnName)){
            JOptionPane.showMessageDialog(parentComponent:null, message:"Indicator Already Present In Table");
            hasduplicate = true;
        }
    }
}

```

If statements check and address errors that come about, during the generation of the table. This is to, allow for a consistent experience.

Populating the Table


```

//Checks if error states are activated
if(!hasduplicate && !endYearError) {

    //Initializes Time column if it doesn't exist
    if (numberOfColumns == 0){
        tblModel.addColumn(columnName: "Time");
        selectedyear = endyear;
    }
    //Initializes Row Count
    for (int rowIndex = 0; rowIndex <= yeardifference; rowIndex++) {
        tblModel.addRow(new Object[] { "", ""});
        String newData = Integer.toString(endyear - i);
        tblModel.setValueAt(aValue: newData, row:rowIndex, tblModel.getColumnCount()-1);
        i = i + 1;
    }
}

// Obtains the data, and stores it in an array
while( startyear <= endyear){
    yearstr = Integer.toString(i: endyear);
    String newData = DataString(collection: "Indicators", country, series: selectedIndicator, year: yearstr);
    newColumnData.add(e: newData);
    endyear = endyear - 1;
}

//Loops through the array list, and rows, and sets the corresponding index value of the rows as that of the array list
tblModel.addColumn(columnName: newColumnName);
for (int rowIndex = 0; rowIndex < tblModel.getRowCount(); rowIndex++) {
    String newData = newColumnData.get(index: rowIndex);
    tblModel.setValueAt(aValue: newData, row:rowIndex, tblModel.getColumnCount() - 1);
}
}

```

First If statement checks if error states are activated before, populating the table. The time column is initialized if it isn't present, at the same time as the rows. The chosen data of the user is obtained using the 'DataString()' method and loops a set number of times('yeardifference'). It's then stored in an arraylist, after which, the array list is looped through to populate the table.

Table and Graph

Select Indicator: GDP per capita

Select Start Year: 2017 Select end Year: 2022

Add Data

Undo

Clear

Generate Line Chart

Generate Bar Chart

Download Line Chart

Download Bar Chart

Time	GDP per capita(India)
2022	2,085.12
2021	1,961.96
2020	1,813.53
2019	1,944.31
2018	1,891.14
2017	1,795.91

Chart Generation

Bar Chart

```

public DefaultCategoryDataset convertTableBar(JTable table) {
    DefaultCategoryDataset dataset = new DefaultCategoryDataset();

    int rowCount = table.getRowCount();
    int columnCount = table.getColumnCount();

    //Extracts Data From Columns
    for (int col = 1; col < columnCount; col++) {
        String category = table.getColumnName(column: col);
        for (int row = 0; row < rowCount; row++) {
            // Gets the year from the first coloum as a string
            String timeString = (String) table.getValueAt(row, column: 0);
            double time;

            try {
                time = Double.parseDouble(s: timeString);
            } catch (NumberFormatException e) {
                // If there's an invalid time value, it's automatically set to zero
                time = 0.0;
            }

            // Looks at each row, and removes commas after the time column

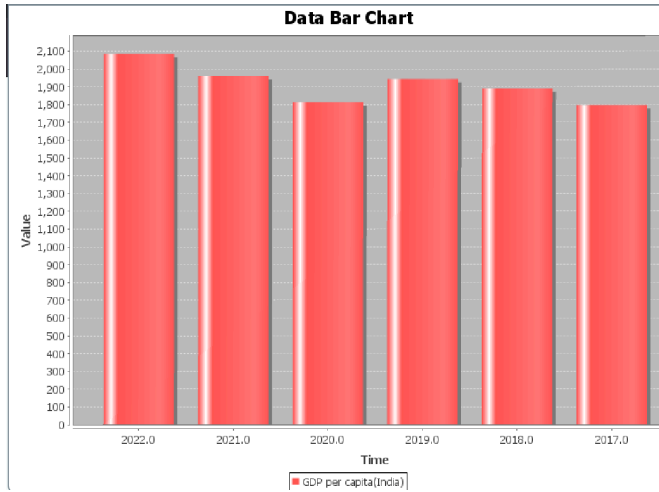
            String valueString = (String) table.getValueAt(row, column: col);
            valueString = valueString.replaceAll(regex: ",", replacement: ""); // Remove commas
            double value;

            try {
                // Converts string to double
                value = Double.parseDouble(s: valueString);
            } catch (NumberFormatException e) {
                // If there is an invalid indicator value, it's automatically set to zero
                value = 0.0;
            }

            dataset.addValue(value, rowKey: category, columnKey: Double.toString(d: time));
        }
    }
    return dataset;
}

```

Here, data from the generated table, is transformed into a dataset. The **JFreeChart** Library is used. The String values of the table are parsed into doubles, and inputted into the dataset. ‘**Try-Catch**’ statement is used to catch exceptions, if there is a string value that cannot be parsed, it automatically set’s it to 0.0, instead of returning an error.



Line Chart

```

public TimeSeriesCollection convertTableLine(JTable table){

    TimeSeriesCollection dataset = new TimeSeriesCollection(); //Initializes time dataset
    //Gets table two and column count
    int rowCount = table.getRowCount();
    int columnCount = table.getColumnCount();

    for (int col = 1; col < columnCount; col++) {
        String category = table.getColumnName(column: col);
        TimeSeries timeSeries = new TimeSeries(name: category);

        for (int row = 0; row < rowCount; row++) {
            // Gets the year from the first coloum as a string
            String yearString = (String) table.getValueAt(row, column: 0);

            if (yearString.equals(anObject: "N/A")) {
                // Skip rows with "N/A" in the year column
                continue;
            }
            int year;
            try {
                year = Integer.parseInt(s: yearString);
            } catch (NumberFormatException e) {
                // Handle or skip the row with invalid year value
                continue;
            }

            // Looks at each row, and removes commas after the time column
            String valueString = (String) table.getValueAt(row, column: col);
            valueString = valueString.replaceAll(regex: ",", replacement:""); // Remove commas
            if (valueString.equals(anObject: "N/A")) {
                // Skip rows with "N/A" in the value column
                continue;
            }
            double value;
            try {
                value = Double.parseDouble(s: valueString);
            } catch (NumberFormatException e) {
                // Handle or skip the row with non-numeric value
                continue;
            }
            // Adds year, value pair to the time series
            timeSeries.addOrUpdate(new Year(year), value);
        }
        dataset.addSeries(series: timeSeries);
    }
    return dataset;}

```

The same concept is used for the line chart, in which a time-series data is used. I chose to use this over a regular line chart, because, timeseries charts are better at handling data gaps, as the **JFreeChart** library automatically, “skips” or adjusts the chart to fit the value rather than leaving a gap.

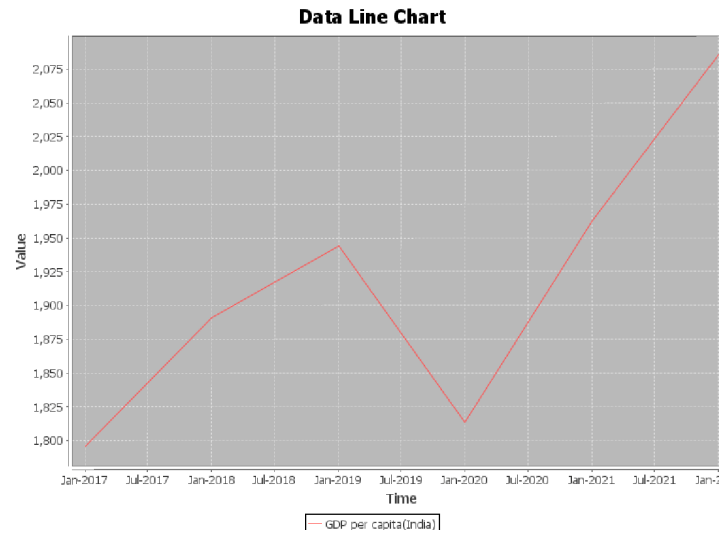


Chart Initialization

```

if (jTable.getColumnCount() >= 2) {
    TimeSeriesCollection dataset = convertTableLine(table: jTable);

    JFreeChart chart = ChartFactory.createTimeSeriesChart(
        title: "Data Line Chart",
        timeAxisLabel: jTable.getColumnName(column: 0),
        valueAxisLabel: "Value",
        dataset,
        legend: true,
        tooltips: true,
        urls: false);
}

```

Initializing of chart using '**convertTableLine()**' method

Saving Charts as PNG

```

while (!saved) {
    String fileName = JOptionPane.showInputDialog(parentComponent:null, message:"Enter A Name To Save The Chart As");

    if (fileName == null) {
        // User canceled the operation
        break;
    }

    if (fileName != null && !fileName.trim().isEmpty()) {

        if (!fileName.endsWith(suffix: ".png")) {
            fileName += ".png";
        }

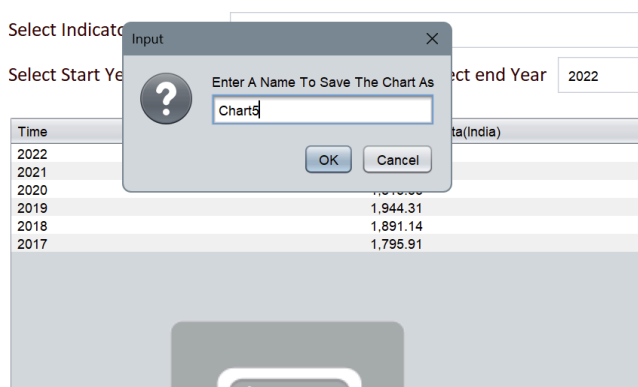
        // Create a file
        File file = new File("C:/Users/twumn/OneDrive/Desktop/School/WORLDBANKIA/src/main/java/Chart/" + fileName);

        if (file.exists()) {
            // Checks if file with the same name already exists
            int option = JOptionPane.showConfirmDialog(parentComponent:null, message:"File already exists. Do you want to rename it?",
                title: "File Exists", optionType: JOptionPane.YES_NO_OPTION);

            if (option == JOptionPane.YES_OPTION) {
                // User wants to rename
                continue; // Continue the loop to ask for a new name
            } else {
                // User chose not to rename, operation is cancelled.
                break;
            }
        } else {
            try {
                ChartUtilities.saveChartAsPNG(file, chart, width: 800, height: 600);
                saved = true; // Sets saved state to true
            } catch (IOException ex) {
                Logger.getLogger(name: Dashboard.class.getName()).log(level: Level.SEVERE, msg:null, thrown: ex);
                JOptionPane.showMessageDialog(parentComponent:null, message:"Error Saving File");
            }
        }
    }
}
} else {
    JOptionPane.showMessageDialog(parentComponent:null, message:"Data Cannot be Converted to Chart"); // If data is lacking the right number of
}
}

```

Here, the code handles user input, from ‘**JOptionPane**’, to let the user decide the name for their selected chart. ‘**while**’ loops are adapted for flow control. ‘**File**’ class, allows the chart to be saved as well as define a path. ‘**try blocks**’ are used to catch exceptions and errors that occur when saving the file.



Choosing Chart Name

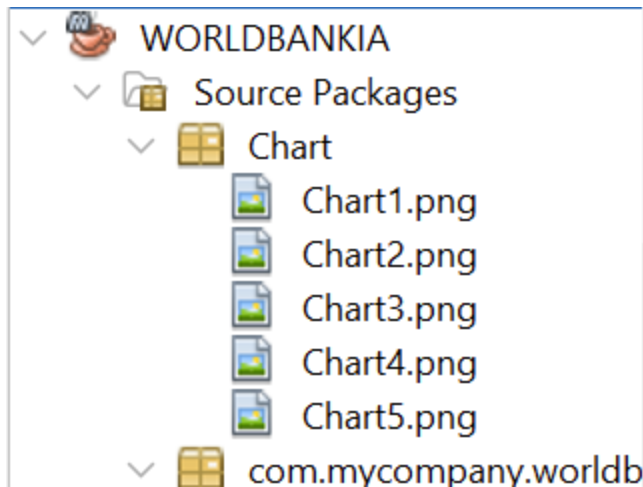


Chart Save Location

Table Manipulation

```

public static void UndoTable(JTable table){
    int numberOfColumns = table.getModel().getColumnCount();
    DefaultTableModel tb1Model = (DefaultTableModel) table.getModel();
    if (numberOfColumns > 1){
        tb1Model.setColumnCount(numberOfColumns - 1);
    }
}

public static void ClearTable(JTable table){
    DefaultTableModel tb2Model = (DefaultTableModel) table.getModel();
    tb2Model.setColumnCount(columnCount:0);
    tb2Model.setRowCount(rowCount:0);
}

```

‘Undo()’ and ‘ClearTable()’ used to allow the user, to fix mistakes they may have made.

GUI Manipulation

```

public void setLabel(JLabel label, String series, String unitFront, String unitBack){

    try {
        label.setText(unitFront + DataString(collection: "Indicators",country,series, year:"2022") + unitBack );

    } catch (IOException ex) {
        Logger.getLogger(name: Dashboard.class.getName()).log(level: Level.SEVERE, msg:null, thrown: ex);
    } catch (GeoIp2Exception ex) {
        Logger.getLogger(name: Dashboard.class.getName()).log(level: Level.SEVERE, msg:null, thrown: ex);
    }

}

```

'setLabel()' accepts **'JLabel'** as a parameter, with which it's value is changed by to the value obtained by the **'DataString()'** class.

```

public void setText() throws GeoIp2Exception, IOException{
    CountryLabel.setText(text: country);

    setLabel(label: PopVar, series: "Total Population",unitFront: "", unitBack: "");

    setLabel(label: InfVar, series: "Yearly Inflation", unitFront: "",unitBack: "%");

    setLabel(label: CapVar, series: "Total Population", unitFront: "",unitBack: "");

    setLabel(label: GDPVar, series: "GDP per capita", unitFront: "$",unitBack: "");

    Object Capital = GetCapital(country);
    CapVar.setText(text: Capital.toString());

    jLabel13.setText(text: country);

}

```

setText() is used to change the Dashboard labels, when there has been a change in the country. I created this method, to easily change all the labels and improve code readability.



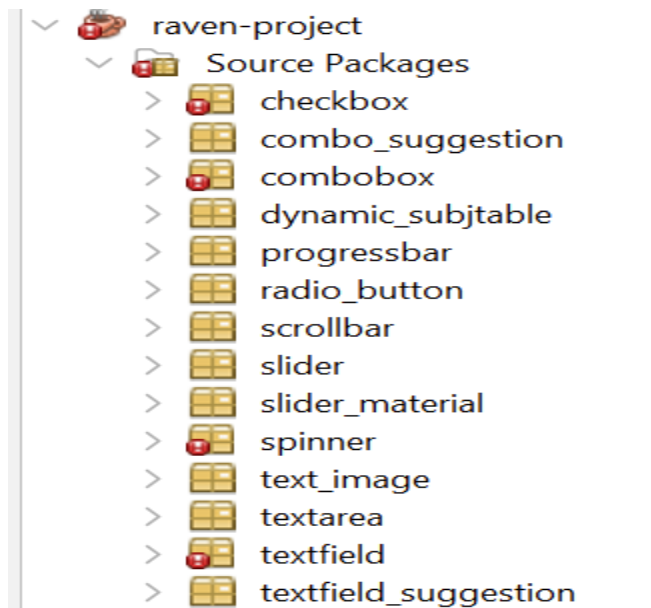
Country



Capital

Angola

Handles, changes in icons, when an action is performed. First 2, scale icon images to fit their set size.



External code for UI elements.

Libraries.

GeoIP2 - <https://www.maxmind.com/en/geoip-demo>

MongoDB - <https://www.mongodb.com/>

JFreeChart - <https://www.jfree.org/jfreechart/>

Bibliography

DJ-Raven - Overview. GitHub. (n.d.). <https://github.com/DJ-Raven>

GEOIP2 and geolite2 database documentation. MaxMind Developer Portal. (n.d.).
<https://dev.maxmind.com/geoip/docs/databases>

Java tutorial: Learn java programming - javatpoint. www.javatpoint.com. (n.d.).
<https://www.javatpoint.com/java-tutorial>

JFreeChart tutorial. Online Tutorials, Courses, and eBooks Library. (n.d.).
<https://www.tutorialspoint.com/jfreechart/>

MongoDB. What is MongoDB? - MongoDB Manual. (n.d.).
<https://www.mongodb.com/docs/manual/>